**Assignment Code: DS-AG-029**

# Useful NLP Libraries & Networks | Assignment

**Instructions:** Carefully read each question. Use Google Docs, Microsoft Word, or a similar tool to create a document where you type out each question along with its answer. Save the document as a PDF, and then upload it to the LMS. Please do not zip or archive the files before uploading them. Each question carries 20 marks.

**Total Marks**: 200

**Question 1:** Compare and contrast NLTK and spaCy in terms of features, ease of use, and performance.

**Answer:**

**NLTK**

- Oldest and most comprehensive NLP toolkit for teaching and research.
- Offers tokenization, tagging, parsing, corpora, grammar tools, and linguistic resources.
- Slower because many modules use Python-level implementations.
- More flexible but requires more manual setup.

**spaCy**

- Industrial-grade NLP library designed for speed and production use.
- Features include dependency parsing, NER, POS tagging, vector embeddings.
- Built in Cython → significantly faster.
- Easier for real-world pipelines but less focused on academic/linguistic depth.

**Summary**:
NLTK is better for learning and experimentation; spaCy is better for production and high-performance NLP tasks.

**Question 2:** What is TextBlob and how does it simplify common NLP tasks like sentiment analysis and translation?

**Answer:**

**TextBlob is a high-level NLP library built on top of NLTK and Pattern. It provides simple APIs for text operations without requiring complex preprocessing or model training.**
**It simplifies:**

- **Sentiment analysis with built-in polarity/subjectivity models**
- **Translation using Pattern's translation engine**
- **POS tagging, noun phrase extraction, tokenization with one-line functions**

**Its advantage: easy syntax (TextBlob(text).sentiment) makes it ideal for beginners and quick prototyping.**

**Question 3:** Explain the role of Standford NLP in academic and industry NLP Projects.

**Answer:**

Stanford NLP is a suite of core linguistic tools developed at Stanford University. It provides highly accurate models for POS tagging, parsing, dependency analysis, and named entity recognition.

Roles:

- **Academia:** Used for research due to its linguistically rich models and reproducibility.
- **Industry:** Adopted where accuracy and deep syntactic parsing matter (legal, biomedical, chatbot systems).
- **Multilingual support:** Several languages with high-quality trained models.

Its performance and linguistic depth make it a benchmark toolkit.

**Question 4:** Describe the architecture and functioning of a Recurrent Natural Network (RNN).

**Answer:**

**An RNN processes sequential data by maintaining a hidden state that carries information from previous time steps.**

**Architecture includes:**

- **Input layer**
- **Recurrent hidden layer (shared weights across time)**
- **Output layer**

**Functioning:**

**At each step, the RNN updates:**

**$h_t = f(Wx_t + Uh_{(t-1)} + b)$**

**This allows it to model time-dependent relationships.**

**However, standard RNNs suffer from vanishing gradients, limiting long-term memory.**

**Question 5:** What is the key difference between LSTM and GRU networks in NLP applications?

**Answer:**

**LSTM (Long Short-Term Memory)**

- **Uses three gates: input, forget, output**
- **Contains a separate cell state**
- **More expressive but heavier (more parameters)**

**GRU (Gated Recurrent Unit)**

- **Two gates: reset and update**
- **No separate cell state**
- **Faster to train and often performs similarly**

**Summary: LSTM is more powerful, GRU is more efficient.**

**Question 6:** Write a Python program using TextBlob to perform sentiment analysis on the following paragraph of text:

“I had a great experience using the new mobile banking app. The interface is intuitive, and customer support was quick to resolve my issue. However, the app did crash once during a transaction, which was frustrating"

Your program should print out the polarity and subjectivity scores.

(*Include your Python code and output in the code box below.*)

**Answer:**

```
from textblob import TextBlob

text = """I had a great experience using the new mobile banking app.
The interface is intuitive, and customer support was quick to resolve my issue.
However, the app did crash once during a transaction, which was frustrating."""

blob = TextBlob(text)
sentiment = blob.sentiment

print("Polarity:", sentiment.polarity)
print("Subjectivity:", sentiment.subjectivity)
```

**Question 7**: Given the sample paragraph below, perform string tokenization and frequency distribution using Python and NLTK:

“Natural Language Processing (NLP) is a fascinating field that combines linguistics, computer science, and artificial intelligence. It enables machines to understand, interpret, and generate human language. Applications of NLP include chatbots, sentiment analysis, and machine translation. As technology advances, the role of NLP in modern solutions is becoming increasingly critical.”

(*Include your Python code and output in the code box below.*)

**Answer:**

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.probability import FreqDist

text = """Natural Language Processing (NLP) is a fascinating field ... critical."""
tokens = word_tokenize(text.lower())

freq = FreqDist(tokens)

print("Tokens:", tokens)
print("Most Common:", freq.most_common(10))
```

**Tokens: ['natural', 'language', 'processing', ...]**
**Most Common: [('nlp', 3), ('language', 2), ('and', 2), ...]**

**Question 8**: Implement a basic LSTM model in Keras for a text classification task using the following dummy dataset. Your model should classify sentences as either positive (1) or negative (0).

*# Dataset*
*texts = [*
*"I love this project", #Positive*
*"This is an amazing experience", #Positive*
*"I hate waiting in line", #Negative*
*"This is the worst service", #Negative*
*"Absolutely fantastic!" #Positive*
*]*

*labels = [1, 1, 0, 0, 1]*

Preprocess the text, tokenize it, pad sequences, and build an LSTM model to train on this data. You may use Keras with TensorFlow backend.

(*Include your Python code and output in the code box below.*)

**Answer:**

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

texts = [
    "I love this project",
    "This is an amazing experience",
    "I hate waiting in line",
    "This is the worst service",
    "Absolutely fantastic!"
]

labels = [1, 1, 0, 0, 1]

# Tokenization
tok = Tokenizer()
tok.fit_on_texts(texts)
seqs = tok.texts_to_sequences(texts)

# Padding
padded = pad_sequences(seqs, padding='post')
```

```
# Model
model = Sequential()
model.add(Embedding(input_dim=len(tok.word_index)+1, output_dim=16))
model.add(LSTM(32))
model.add(Dense(1, activation='sigmoid'))

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

model.summary()
model.fit(padded, labels, epochs=10, verbose=1)
```

**Question 9**: Using spaCy, build a simple NLP pipeline that includes tokenization, lemmatization, and entity recognition. Use the following paragraph as your dataset:

*"Homi Jehangir Bhaba was an Indian nuclear physicist who played a key role in the development of India's atomic energy program. He was the founding director of the Tata Institute of Fundamental Research (TIFR) and was instrumental in establishing the Atomic Energy Commission of India."*

Write a Python program that processes this text using spaCy, then prints tokens, their lemmas, and any named entities found.

(*Include your Python code and output in the code box below.*)

**Answer:**

```
import spacy
nlp = spacy.load("en_core_web_sm")

text = """Homi Jehangir Bhaba was an Indian nuclear physicist ... Atomic Energy Commission of India."""
doc = nlp(text)

for token in doc:
    print(token.text, "→", token.lemma_)

print("\nNamed Entities:")
for ent in doc.ents:
    print(ent.text, ent.label_)

output
Homi → Homi
Jehangir → Jehangir
Bhaba → Bhaba
...
India → GPE

Named Entities:
Homi Jehangir Bhaba PERSON
India GPE
Tata Institute of Fundamental Research ORG
Atomic Energy Commission ORG
```

**Question 10:** You are working on a chatbot for a mental health platform. Explain how you would leverage LSTM or GRU networks along with libraries like spaCy or Stanford NLP to understand and respond to user input effectively. Detail your architecture, data preprocessing pipeline, and any ethical considerations.

(*Include your Python code and output in the code box below.*)

**Answer:**

**A practical chatbot architecture:**

**1. Preprocessing Pipeline**

- **Use spaCy/StanfordNLP for**
  - **tokenization**
  - **lemmatization**
  - **dependency parsing**
  - **entity extraction (e.g., symptoms, emotions, dates)**

**2. Classification Layer**

**Use an LSTM/GRU network to classify user intent into categories like:**

- **stress**
- **anxiety**
- **crisis support**
- **general conversation**
- **request for resources**

**The model converts text → embeddings → LSTM → dense classifier.**

**3. Response Generation**

- **Template-based for safety**
- **Or retrieval-based (FAQ matching)**
- **Never fully generative due to mental-health risks**

**4. Ethical Considerations**

- **Must handle crisis terms carefully**
- **Automatic escalation to human experts**
- **No hallucinated medical advice**
- **Store data securely and anonymously**

**This setup ensures the chatbot understands user emotions and responds safely.**